



EE6008 - Deep Learning at Edge Project

Real-Time Drone Hunter (OpenMV - RT1062)

Team Members

Siddartha Sandeep P

Rajarshi Karmakar

Deepend Pradeep

Chandana Bakam

Neil Shaun Dias

MEng in Computer Vision and Artificial Intelligence

MEng in Edge Computing

Deep Learning Project Report

Module Leader: **Dr. Brendan Mullane**

Department of Electronics and Computer Engineering

Contents

1	Abstract	4
2	Introduction	4
3	Motivation	4
4	Environment Setup and Target Device Configuration	4
5	Dataset Selection	5
6	Model Tuning with EON Tuner	6
7	Model Customisation and Training	7
7.1	Training Configuration	7
7.2	Training Results and Metrics	8
7.3	Validation Results	9
8	Model Testing and Evaluation	9
8.1	Test Results	9
8.2	Feature Explorer Visualization	10
8.3	Visual Predictions on Test Set	10
9	User Interface and Real-time Detection Dashboard	11
9.1	Interface Overview	11
9.2	Backend Functionality	11
9.3	Deployment Screenshot	12
10	Challenges Faced and Future Scope	12
10.1	Challenges During Development	12
10.2	Future Improvements	13
11	References	13
12	Github Repository	14

List of Figures

1	Feature Explorer showing distribution of the drone class. .	5
2	Edge Impulse EON Tuner configurations and their confusion matrices. Best-performing models (top row) use the squash and fit-long resize modes with 85% accuracy.	6
3	Validation metrics across training epochs	8
4	Training vs Validation loss per epoch	8
5	Feature Explorer: Correct vs. Incorrect Detections on Test Set	10
6	Visual drone detections with confidence scores on test set images	10
7	Drone detections rendered in the real-time dashboard (OpenMV RT1062 firmware output)	12

List of Tables

1	Target Device Configuration: OpenMV-RT1062	5
2	Train/Test Split	5
3	Comparison of EON Tuner model variants using different input resize modes and hyperparameters.	7
4	Validation Set Metrics	9
5	Validation Confusion Matrix	9

1 Abstract

This project focuses on building a real-time drone detection system using the OpenMV RT1062 microcontroller. A curated drone dataset was trained using the FOMO architecture in Edge Impulse. The model was optimized, quantized, and deployed to the RT1062. A Python GUI was also created to show live detection results. The system achieved over 86% accuracy on the test set and highlights practical deployment of embedded AI.

2 Introduction

"Drone Hunter using RT1062" is a project aimed at creating a real-time drone detection solution on a resource-constrained device. Using the OpenMV Cam RT1062 with the NXP i.MX RT1062 chip, it leverages Edge Impulse's ML pipeline to train and deploy a lightweight FOMO object detection model. The setup allows drone detection directly on the device, suitable for edge-based monitoring without cloud dependency.

3 Motivation

Drones pose a growing risk in secure areas, yet traditional detection systems are costly or inefficient for field deployment. This project is driven by the need for a low-cost, energy-efficient, real-time detection system that can run fully on-device. By targeting the RT1062 microcontroller, we demonstrate that embedded AI can support modern security and surveillance use cases effectively.

4 Environment Setup and Target Device Configuration

The deployment target for this project is the OpenMV RT1062, which features a 600MHz ARM Cortex-M7 processor. It has 32MB of RAM and 16MB of Flash memory, with no dedicated accelerator. These constraints guided model selection and optimization to ensure efficient edge performance.

The application requires a maximum inference latency of 100ms, suitable for real-time drone detection. These specifications were configured in Edge Impulse to simulate realistic deployment behavior.

Table 1: Target Device Configuration: OpenMV-RT1062

Parameter	Value
Processor Family	ARM Cortex-M
Processor Architecture	Cortex-M7
Clock Speed	600 MHz
Accelerator	None
RAM Budget	32,000 KB
ROM Budget	16,000 KB
Maximum Inference Latency	100 ms
Device Name (Custom)	OpenMV - RT1062

5 Dataset Selection

A custom drone object detection dataset consisting of **2,034 labeled images** was curated from open online sources. Poor quality and irrelevant samples were removed to ensure effective training. The dataset was split into **82% training** and **18% testing** sets.

Table 2: Train/Test Split

Split	Number of Images
Training	1,655
Testing	379

To balance accuracy and performance, three image resolutions were tested: **96×96**, **128×128**, and **160×160**. The 96×96 size was chosen for its optimal trade-off between performance and resource usage on the RT1062.

Feature Explorer:

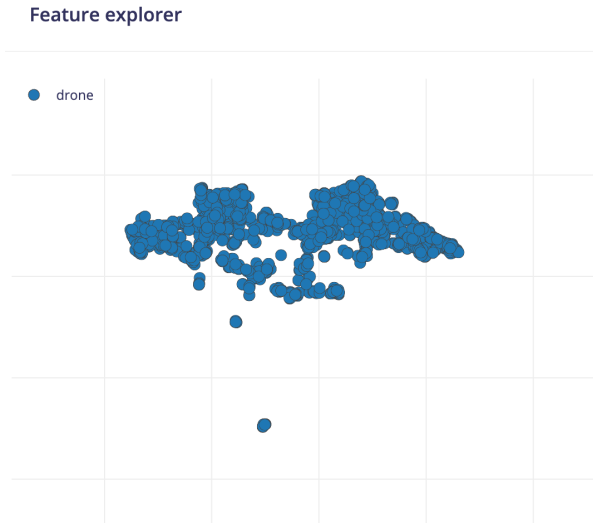


Figure 1: Feature Explorer showing distribution of the drone class.

6 Model Tuning with EON Tuner

To optimize the performance of the object detection model for deployment on the OpenMV RT1062, the EON Tuner tool from Edge Impulse was used. This tool allows automatic exploration of various model architectures, image input dimensions, resize modes, and training hyperparameters.

The objective was to strike a balance between accuracy, latency, and memory usage to ensure the model performs well under the device constraints. The dataset used for tuning contained 2,034 images of drones, with an 82%/18% train-test split. Input images were grayscale with a resolution of 96×96 , and several resize modes such as squash, fit-short, and fit-long were tested.



Figure 2: Edge Impulse EON Tuner configurations and their confusion matrices. Best-performing models (top row) use the squash and fit-long resize modes with 85% accuracy.

The EON Tuner tested various MobileNetV2-based FOMO (Fast Object Detection) models. A summary of the top configurations and their performance is shown in Table 3. The best-performing model in terms of accuracy and efficiency was the one trained using the *squash* resize mode, achieving 85% accuracy and an F1 score of 0.85 with a latency of 10 ms.

Model ID	Resize Mode	Accuracy (%)	F1 Score	Latency (ms)	RAM / ROM (KB)
grayscale-fomo-731	Squash	85	0.85	10	239 / 79
grayscale-fomo-36e	Fit-long	85	0.85	8	239 / 79
grayscale-fomo-e5a	Squash	83	0.83	14	239 / 79
grayscale-fomo-8c4	Fit-short	81	0.81	11	239 / 79
grayscale-fomo-672	Fit-long	81	0.81	11	239 / 79
grayscale-fomo-1e9	Fit-long	80	0.80	13	239 / 79
grayscale-fomo-461	Squash	79	0.79	11	239 / 79
grayscale-fomo-48e	Fit-long	76	0.76	11	239 / 79

Table 3: Comparison of EON Tuner model variants using different input resize modes and hyperparameters.

7 Model Customisation and Training

Following the model selection from EON Tuner, we chose the *grayscale-fomo-731* model due to its consistent performance using the *Squash* resize mode. To further enhance its accuracy, we customised and continued training this model using the Edge Impulse Neural Network (NN) block and exported the training configuration to a custom training script.

7.1 Training Configuration

The training was performed with the following settings:

- **Model Architecture:** FOMO MobileNetV2 0.35
- **Input Shape:** 96x96 grayscale images (9,216 features)
- **Training Cycles:** 35
- **Learning Rate:** 0.003 (Adam optimizer)
- **Batch Size:** 32
- **Validation Split:** 20%
- **Augmentation:** Enabled (rotation, flip, contrast)
- **Int8 Quantization:** Enabled for deployment

The model was trained using a custom Python script which leveraged TensorFlow and Edge Impulse libraries for constrained object detection. The architecture retained the MobileNetV2 backbone up to `block_6_expand_relu` and added a lightweight detection head, outputting bounding box logits across spatial dimensions.

7.2 Training Results and Metrics

The training process showed steady improvements across key performance metrics. Figure 3 displays the evolution of validation precision, recall, and F1-score throughout the training epochs. The F1 score stabilised at approximately 0.86, indicating a good balance between precision and recall.



Figure 3: Validation metrics across training epochs

Figure 4 shows the training and validation loss across all 35 epochs. The losses follow a decreasing trend, indicating effective learning.

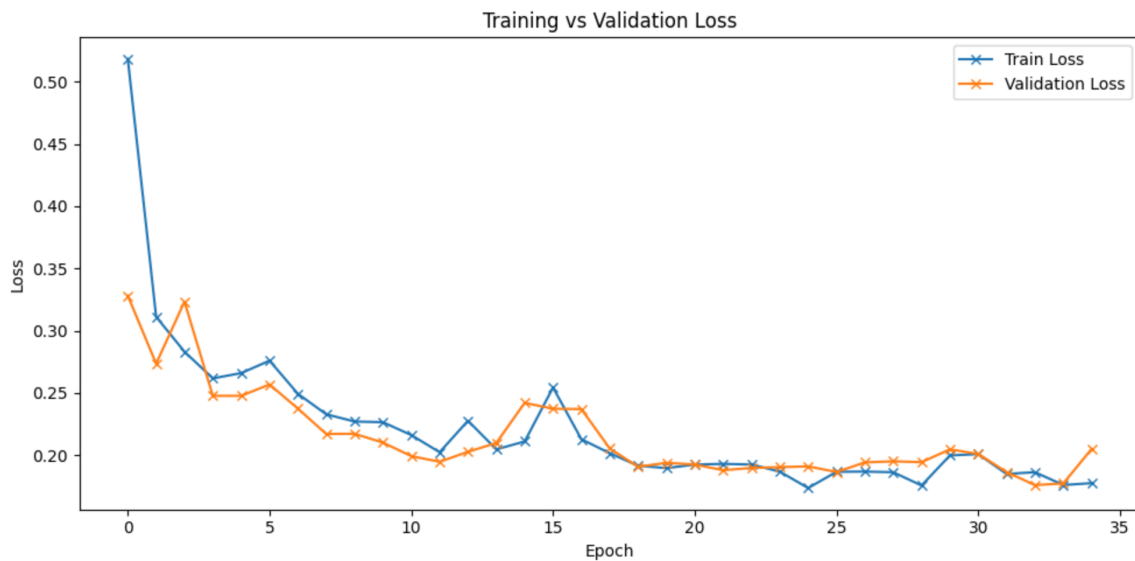


Figure 4: Training vs Validation loss per epoch

7.3 Validation Results

Table 4 summarises the final model metrics on the validation set.

Table 4: Validation Set Metrics

Metric	Value
Precision (non-background)	0.83
Recall (non-background)	0.90
F1 Score (non-background)	0.86

The confusion matrix in Table 5 demonstrates that the model is capable of distinguishing drones from background accurately, with minimal false negatives.

Table 5: Validation Confusion Matrix

	Pred: Background	Pred: Drone
True: Background	100.0%	0.0%
True: Drone	9.7%	90.3%

Overall, the custom-trained FOMO model provided an excellent trade-off between model size, speed, and accuracy, making it suitable for deployment on the RT1062 embedded platform.

8 Model Testing and Evaluation

To assess generalization performance, the trained FOMO model was evaluated on a held-out test set containing 379 images. The test set was curated to include diverse drone scenarios similar to real-world conditions. The same preprocessing pipeline (96x96 grayscale, squash resize mode) was applied to ensure consistency with training.

8.1 Test Results

The model achieved strong performance on the test set, closely mirroring validation results. The following metrics were obtained:

- **Test Accuracy:** 86.02%
- **Test Precision (non-background):** 0.87
- **Test Recall (non-background):** 0.98
- **Test F1 Score (non-background):** 0.92

These results confirm the model’s robustness and ability to accurately detect drones across varied scenarios.

8.2 Feature Explorer Visualization

Figure 5 presents a snapshot from the Feature Explorer tool, highlighting how the model distinguishes between correct and incorrect object detection predictions. Correct predictions are clustered well, while misclassified samples appear scattered.

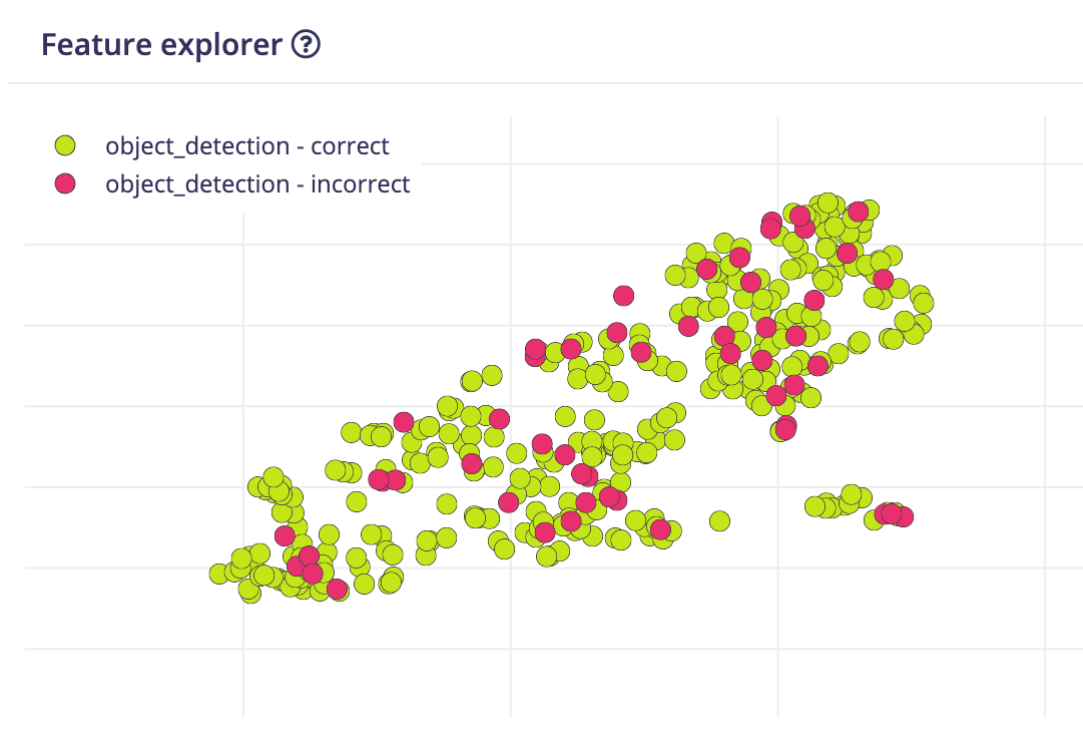


Figure 5: Feature Explorer: Correct vs. Incorrect Detections on Test Set

8.3 Visual Predictions on Test Set

Qualitative evaluation was also conducted by visualizing model predictions on test images. Bounding boxes were drawn over detected drones, with class probabilities. Figure 7 presents two test images side-by-side to demonstrate the model's object detection capability.



Figure 6: Visual drone detections with confidence scores on test set images

9 User Interface and Real-time Detection Dashboard

To provide real-time visual feedback from the deployed drone detection model on the OpenMV RT1062, a Python-based desktop interface was developed using Tkinter. The interface communicates with the microcontroller over a serial USB connection to visualize detections.

9.1 Interface Overview

The dashboard displays live detection status, logs all timestamps of detected drones, and shows a real-time snapshot from the camera with detection heatmaps overlaid.

- **Status display:** Indicates whether a drone is currently detected.
- **Detection counter:** Tracks the total number of detections during the session.
- **Log window:** Stores detection timestamps.
- **Snapshot viewer:** Shows real-time images with detection overlays.

9.2 Backend Functionality

The backend listens to the serial port for messages from the OpenMV RT1062. When a "DRONE DETECTED" message is received, the software:

1. Parses detection coordinates and confidence scores.
2. Reads a JPEG snapshot from the device.
3. Generates a red heatmap based on detection intensity.
4. Overlays the heatmap on the camera snapshot.
5. Updates the GUI with this processed image and logs the event.

9.3 Deployment Screenshot

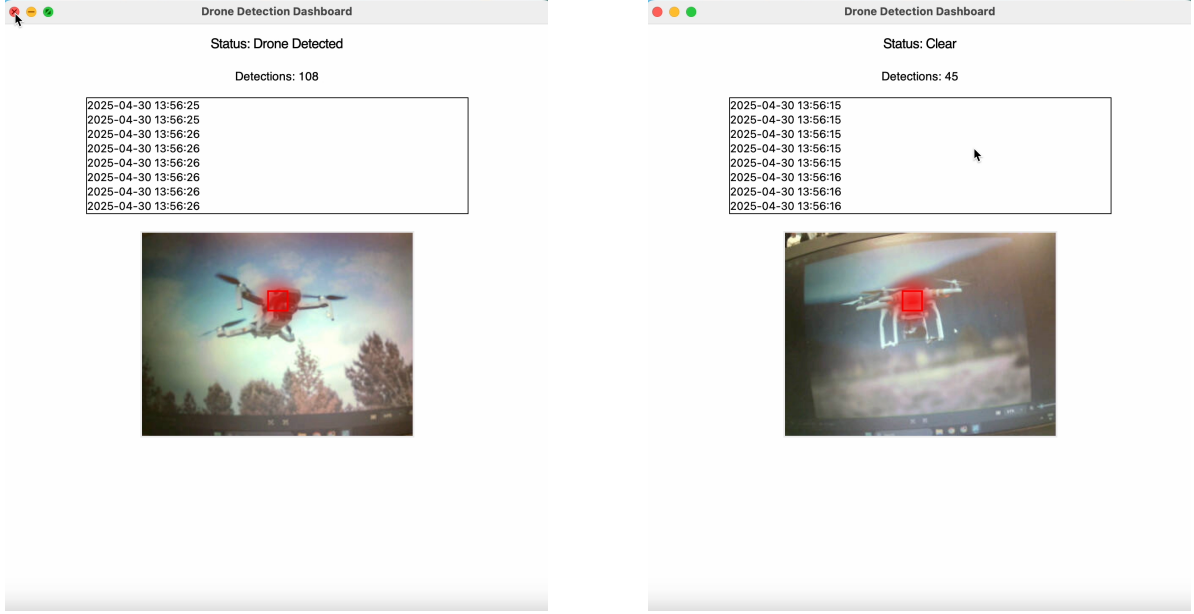


Figure 7: Drone detections rendered in the real-time dashboard (OpenMV RT1062 firmware output)

10 Challenges Faced and Future Scope

10.1 Challenges During Development

The journey from dataset preparation to real-time deployment presented several technical and practical challenges:

- **Fine-tuning pretrained models:** One major challenge was resuming training on a TensorFlow Lite model exported from Edge Impulse. While the original FOMO MobileNetV2 model performed well, local fine-tuning beyond epoch 35 (eg. up to 60 epochs) was attempted to further improve accuracy. However, handling model weight formats and optimizer compatibility in custom scripts proved non-trivial and time-consuming.
- **Local training complexities:** Training the object detection model locally using the exported Edge Impulse pipeline required replicating data augmentation, segmentation map logic, and exact loss formulations. Although partial success was achieved, inconsistencies in augmentation and label encoding occasionally impacted convergence.
- **Compute limitations:** Edge Impulse's free tier limited training time per session to 20 minutes. Many promising hyperparameter setups failed due to exceeding this threshold, requiring multiple manual

restarts and trials. Leveraging EON Tuner helped mitigate some of these issues by automating parameter sweeps.

- **OpenMV integration:** Reading JPEG snapshots over UART from the RT1062 and parsing detection coordinates in sync posed occasional synchronization issues, requiring careful buffer handling in Python and OpenMV firmware to maintain reliable operation.

10.2 Future Improvements

This project lays the foundation for an edge-based drone detection system, with several enhancements planned:

- **Multi-class detection:** Extend detection capabilities to differentiate between drone types or identify other aerial objects (eg. birds, kites).
- **Audio-visual fusion:** Integrate sound classification (eg. drone propeller noise) alongside vision to reduce false positives and boost reliability.
- **GUI enhancements:** Add detection charts, snapshot saving, and alert sound notifications in the Tkinter UI to support real-world monitoring setups.

These efforts would evolve the system into a production-ready, low-power autonomous drone monitoring solution.

Acknowledgment

We would like to express our sincere gratitude to Prof Dr. Brendan Mulane for his valuable guidance, support, and insightful lectures throughout this project. His expertise and feedback were instrumental in the successful completion of this work.

11 References

Edge Impulse. (2024). *Embedded Machine Learning Platform*. Retrieved from <https://www.edgeimpulse.com>

OpenMV. (2024). *OpenMV Cam RT1062 Product Overview*. Retrieved from <https://openmv.io>

NXP Semiconductors. (2022). *i.MX RT1060 Crossover Processors for Consumer Products*. Retrieved from <https://www.nxp.com>

Edge Impulse. (2022). *FOMO: Faster Objects, More Objects*. Retrieved from <https://docs.edgeimpulse.com/docs/edge-impulse-studio/learning-block/object-detection/fomo>

TensorFlow. (2023). *TensorFlow: An end-to-end open-source machine learning platform*. Retrieved from <https://www.tensorflow.org>

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 4510-4520.

12 Github Repository

<https://github.com/siddy-p/Drone-Hunter> (All project related files)

https://github.com/rkarmaka98/drone_detection_edge (Gradio dashboard)

<https://studio.edgeimpulse.com/public/671562/live> (Edge impulse)